

ООП: АБСТРАКТНЫЕ КЛАССЫ

Семинар 4

Чисто виртуальная функция

- объявлена в классе со спецификатором `virtual`;
- объявлена с использованием специального синтаксиса C++, который определяет функцию как не имеющую блока фигурных скобок `{ }` содержащего программный код.

```
1  class ClassName
2  {
3      virtual return_type FuncNamePure(list_of_parameters) = 0;
4  };
```

Абстрактный класс

Абстрактный класс – это класс, содержащий хотя бы одну чисто виртуальную функцию.

Создать объект абстрактного класса не получится. Попытка создания такого объекта будет распознана компилятором как ошибка, так как реализация виртуальной функции отсутствует.

Абстрактный класс

```
class ClassName
{
    virtual return_type Func(list_of_parameters) = 0;

    // Другие элементы класса
    // ...
};
```

ClassName obj; // ошибка компиляции, запрещено

ClassName* p; // объявление указателя на абстрактный класс
ClassName

ClassName& ref = obj; // объявление ссылки на абстрактный класс
ClassName

std::string

- заголовочный файл <string>
https://en.cppreference.com/w/cpp/string/basic_string
- класс с методами и переменными для организации работы со строками в языке программирования C++;
- определение std::basic_string<char>.

```
std::string str = {"Mirea"};
std::string new_str;
std::cout << "new_str = " | << new_str << std::endl;
new_str = str;
std::cout << "new_str = " << new_str << std::endl;

const char* p = new_str.c_str();
std::cout << "p = " << p << std::endl;
```

std::string

C → C++

- strcmp → compare, ==
- strstr, strchr → find
- strlen → length / size
- strcpy → copy
- strcat → append, +

```
compare(strcmp): 0
find(strstr): rea from pos 2
find(strchr): irea from pos 1
length(strlen): 5 5
copy strcpy: Mirea
append(strcat): Mirea 2023
```

```
14 // strcmp
15 int res_compare = str.compare(new_str);
16 std::cout << "compare(strcmp): " << res_compare << std::endl;
17 // strstr, strchr
18 size_t n = new_str.find("rea");
19 if (n != std::string::npos)
20 |     std::cout << "find(strstr): " << new_str.substr(n) <<
21 |     |     |     | " from pos " << n << std::endl;
22
23 n = new_str.find('i');
24 if (n != std::string::npos)
25 |     std::cout << "find(strchr): " << new_str.substr(n) <<
26 |     |     |     | " from pos " << n << std::endl;
27
28 // strlen
29 std::cout << "length(strlen): " << new_str.length() << " " <<
30 |     |     |     | new_str.size() << std::endl;
31
32 // strcpy
33 char scopy[10]{};
34 new_str.copy(scopy, sizeof(scopy) - 1);
35 std::cout << "copy strcpy): " << scopy << std::endl;
36
37 // strcat
38 new_str.append(" 2023");
39 std::cout << "append(strcat): " << new_str << std::endl;
```

Non-member функции

Input/output

<code>operator<<</code> <code>operator>></code>	performs stream input and output on strings (function template)
<code>getline</code>	read data from an I/O stream into a string (function template)

Numeric conversions

<code>stoi</code> (C++11) <code>stol</code> (C++11) <code>stoll</code> (C++11)	converts a string to a signed integer (function)
<code>stoul</code> (C++11) <code>stoull</code> (C++11)	converts a string to an unsigned integer (function)
<code>stof</code> (C++11) <code>stod</code> (C++11) <code>stold</code> (C++11)	converts a string to a floating point value (function)
<code>to_string</code> (C++11)	converts an integral or floating point value to string (function)

operator+

`operator==`
`operator!=` (removed in C++20)
`operator<` (removed in C++20)
`operator>` (removed in C++20)
`operator<=` (removed in C++20)
`operator>=` (removed in C++20)
`operator<=>` (C++20)

std::stringstream

- заголовочный файл `<sstream>`
https://en.cppreference.com/w/cpp/io/basic_stringstream
- `stringstream` – потоковый класс для строк, предоставляющий буфер для хранения данных. В отличие от `cin/cout` не подключен к каналу ввода/вывода;
- все, что выводится (`<<`) в такой поток, добавляется в конец строки;
- все, что считывается (`>>`) из потока – извлекается из начала строки.

Пример

```
1  #include <sstream>
2  #include <iostream>
3
4  int main(int argc, char* argv[])
5  {
6      std::stringstream ss;
7      ss << "123";
8      int k = 0;
9      ss >> k;
10     std::cout << k << std::endl;
11     return 0;
12 }
13
```

→ 123

```
1  #include <iostream>
2  #include <sstream>
3
4  int main(int argc, char** argv)
5  {
6      int example = 1234;
7      std::ostringstream oss;
8      oss << example;
9      std::string temp = oss.str();
10     std::cout << temp << std::endl;
11     return 0;
12 }
13
```

→ 1234

Разбиение строки на токены

```
istream& getline(istream& is, string& str, char delim);      (1)
```

```
istream& getline(istream& is, string& str);                  (2)
```

```
1  ✓ #include <sstream>
2    #include <iostream>
3
4  ✓ int main(int argc, char* argv[])
5  {
6      std::string str = "this,is,text,to,tokenize";
7      std::istringstream ss(str);
8      std::string tok;
9  ✓  while (std::getline(ss, tok, ','))
10     {
11         std::cout << tok << std::endl;
12     }
13     return 0;
14 }
15
```



```
shulginaali@075-AShulgina:~/cpp$ ./a.out
this
is
text
to
tokenize
```

Задачи

Пользователем с клавиатуры задается строка, разделенная точками с запятыми. Записать в вектор каждую подстроку в следующем виде:

<номер подстроки>: модифицированная подстрока

Модифицированная подстрока получается взятием подстроки от последнего символа '/', встречающегося в строке. Если символа нет – берется вся строка.

Вывести полученный результат на экран построчно.

Пример

Ввод: /home/user/text.txt;/etc/sensors.d;/tmp/config.ini;deploy.sh;

Вывод: 0: text.txt

1: sensors.d

2: config.ini

3: deploy.sh

Задачи

Реализовать базовый абстрактный класс фигуры (Shape) со следующими чисто-виртуальными функциями:

- `area()` – подсчет площади;
- `perimeter()` – подсчет периметра;
- `print()` – печать информации о фигуре (тип, параметры, площадь, периметр).

Создать три наследника базового класса: треугольник (Triangle), прямоугольник (Rectangle) и окружность (Circle), реализовать виртуальные функции из базового класса. Приватные поля параметров фигур задать самостоятельно (треугольник – 3 стороны, прямоугольник – 2 стороны, окружность – радиус).

Написать функцию, печатающую информацию о фигурах, содержащихся в `std::vector`.

В функции `main` создать экземпляры каждого унаследованного класса, записать в `std::vector`. Вызвать функцию печати информации.